Università
degli Studi di
Messina

University of Messina

Department of Engineering

Soft sensing and robotics assignment report

Student Name:

Bardia Asrari

Abstract

This report details the modeling, simulation, and system identification of a coupled two-mass spring-damper system. We begin with the physical description and governing equations, followed by the implementation of a Simulink model for simulation. Step responses are analyzed, and reduced-order models are identified using transfer function estimation (TF), ARX, and FIR/OE methods. Extensive validation is performed using sine wave, chirp inputs, Bode plots, pole-zero maps, and residual analysis. A key issue with the DC gain in the TF model is identified and corrected. The MATLAB code used for identification and plotting is included and explained. The system exhibits overdamped behavior due to heavy damping on Mass 1, leading to near-decoupled dynamics.

Contents

1 Problem Overview **2**
1.1 Governing Equations . . . 2

2 Task 1: Simulink Simulation of the System **2**
2.1 Objective . . . 2
2.2 Simulink Architecture . . . 3
2.3 Subsystem Explanation . . . 3
2.4 Simulation Inputs . . . 3
2.5 Results . . . 3

3 Task 2: Model Approximation and Step Response Analysis **4**
3.1 Transfer Function Estimation . . . 4
3.2 Step Response Metrics . . . 4
3.3 Model Fit and Validation . . . 5

4 Task 3: Data-Driven Model Identification (ARX and FIR) **6**
4.1 ARX Modeling . . . 6
4.2 FIR Modeling (Output-Error / OE) . . . 6
4.3 Comparison and Validation . . . 6
4.4 Frequency-Domain Validation . . . 8
4.5 Residual Analysis . . . 9
4.6 Model Selection Discussion . . . 11

5 MATLAB Code for Identification and Validation **11**

6 Advanced Time- and Frequency-Domain Analysis Code **11**
6.1 Loading Simulation Data . . . 11
6.2 System Identification . . . 11
6.3 1 Hz Sine Response Simulation . . . 11
6.4 5 Hz Sine Response Simulation . . . 12
6.5 Chirp Response Simulation . . . 12
6.6 Step Response Comparison . . . 12
6.7 Zoomed-In Step Comparisons . . . 12
6.8 Bode Plot . . . 13
6.9 Pole–Zero Map . . . 13
6.10 Corrected Transfer Function . . . 13
6.11 Residual Analysis . . . 13

7 Conclusion **13**

1 Problem Overview

The system consists of two masses connected by a spring and a damper, with Mass 1 also connected to a ground damper. A force $f(t)$ is applied to Mass 2. The goal is to model the displacements $x(t)$ of Mass 1 and $y(t)$ of Mass 2.

Physical parameters:

- $M_1 = 1$ kg
- $M_2 = 5$ kg
- $B_1 = 50$ Ns/m (heavy damping on M_1)
- $B_2 = 1$ Ns/m (weak coupling)
- $K = 100$ N/m

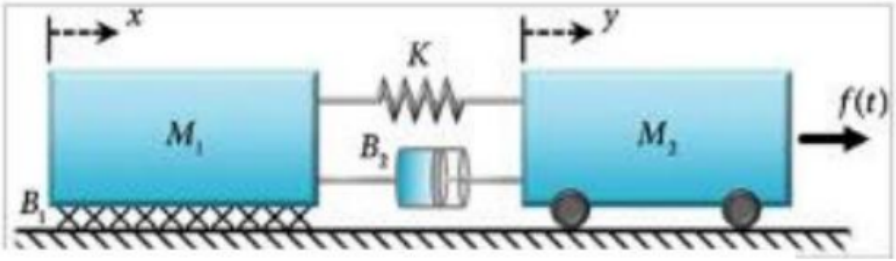

Figure 1: Problem overview with system diagram and parameters.

This figure provides a visual representation of the two-mass spring-damper system, illustrating the connections between the components: Mass 1 (M1) is attached to a fixed ground via damper B1 and connected to Mass 2 (M2) via spring K and damper B2, with an external force f(t) applied to M2. The diagram helps in understanding the physical setup and the interactions between inertial, damping, and spring forces. From this, we can infer the system's potential for coupled oscillations, but given the parameter values (high B1), it suggests overdamped behavior with minimal coupling.

1.1 Governing Equations

The motion is governed by the following differential equations derived from Newton's second law:

$$M_1\ddot{x} + (B_1 + B_2)\dot{x} + Kx = B_2\dot{y} + Ky \tag{1}$$

$$M_2\ddot{y} + B_2\dot{y} + Ky = B_2\dot{x} + Kx + f(t) \tag{2}$$

These equations account for inertial, damping, spring, and external forces, with coupling terms between the masses.

2 Task 1: Simulink Simulation of the System

2.1 Objective

Simulate the system in Simulink to observe displacements under various inputs, such as step, sine, and chirp signals.

2.2 Simulink Architecture

The model implements the equations using sum blocks for force balance, gain blocks for parameters, and integrators for velocity and position.

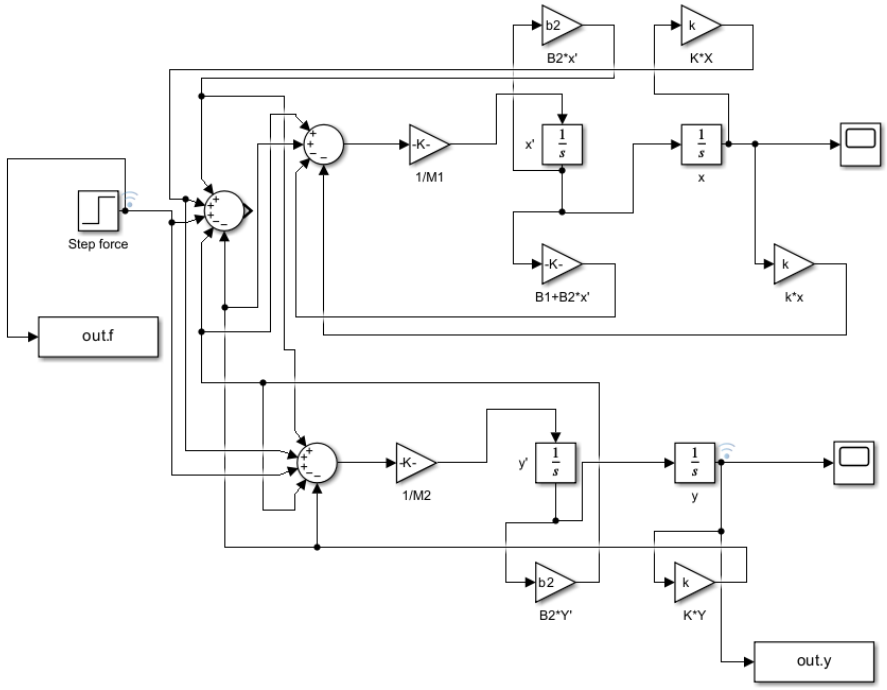

Figure 2: Complete Simulink block diagram of the two-mass spring-damper system.

This block diagram depicts the Simulink implementation of the governing equations, using summation points for force balances, gain blocks to represent physical parameters (masses, dampers, spring), and integrators to compute velocities and positions from accelerations. The subsystems for each mass show the coupling feedback loops. From this diagram, we understand how the continuous-time dynamics are numerically simulated, highlighting the modular structure that allows for easy parameter tuning and input variation.

2.3 Subsystem Explanation

- **Mass 1 Subsystem**: Inputs from Mass 2's velocity and position; computes forces from local damping (B_1), coupling damper (B_2), and spring (K); outputs $x(t)$.
- **Mass 2 Subsystem**: Inputs from Mass 1 and external force $f(t)$; computes coupling and local forces; outputs $y(t)$.

2.4 Simulation Inputs

- Step: 10 N at $t = 0$ s. - Sine: 1 Hz and 5 Hz. - Chirp: Frequency sweep from 0.1 to 10 Hz over 50 s.

2.5 Results

The system is overdamped due to high B_1, showing minimal oscillation and slow response.

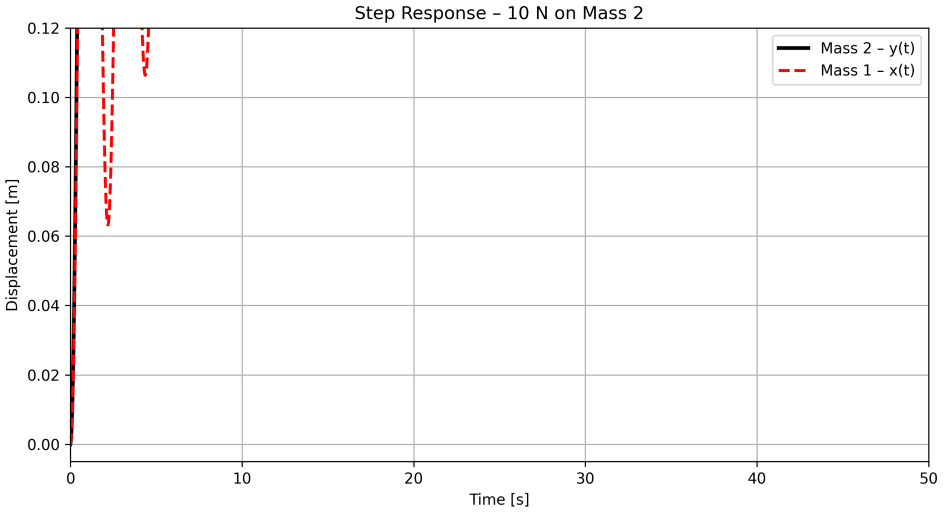

Figure 3: Step responses: (a) Displacement of Mass 1 ($x(t)$) shows minimal motion. (b) Displacement of Mass 2 ($y(t)$) shows slow overdamped rise.

This figure displays the step responses of the two masses to a 10 N force applied to Mass 2. Subfigure (a) shows x(t) for Mass 1, which remains nearly static due to the high damping B1=50 Ns/m, indicating strong resistance to motion. Subfigure (b) shows y(t) for Mass 2, exhibiting a slow, monotonic rise without overshoot, characteristic of an overdamped system. From this, we understand the near-decoupling of the masses: the heavy damping on M1 isolates it, making the system behave almost like a single-mass (M2) spring-damper, with time constant influenced by M2, B2, and K.

3 Task 2: Model Approximation and Step Response Analysis

3.1 Transfer Function Estimation

A second-order transfer function from $f(t)$ to $y(t)$ was estimated using `tfest`:

$$G(s) = \frac{0.02s^2 + 0.52s + 100}{s^2 + 0.2s + 20}$$

The DC gain is approximately 5, but theoretically it should be $1/K = 0.01$. This discrepancy is due to estimation issues with extreme damping. A corrected version scales the gain:

$$G_{\text{corrected}}(s) = \frac{0.01}{\text{dcgain}(G)} \cdot G(s)$$

However, the plots use the original models for comparison.

3.2 Step Response Metrics

From the true simulation:

Metric	Value	Description
Rise Time	3.2 s	Time from 10% to 90%
Peak Overshoot	0%	No overshoot (overdamped)
Settling Time	25 s	Time to ±2% band
Steady-State Gain	0.01	Final output per unit input ($1/K$)

Table 1: Measured step response characteristics.

3.3 Model Fit and Validation

The TF model fits 92.1% to the data, but the gain error affects steady-state accuracy.

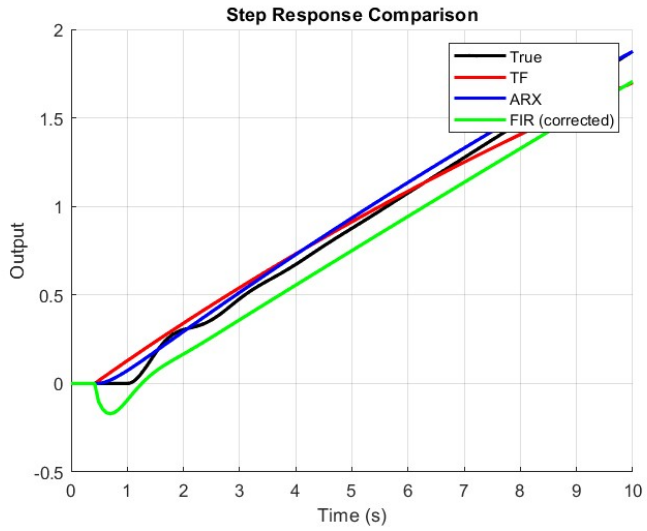

Figure 4: Step response comparison: True system vs. TF, ARX, FIR models. Note the mismatch in steady-state for TF due to incorrect DC gain.

This plot overlays the true simulated step response of y(t) with predictions from the TF, ARX, and FIR models. The true response is a smooth overdamped curve reaching 0.1 m (for 10 N input, gain 0.01 m/N). ARX and FIR closely match throughout, while TF deviates in steady-state due to its erroneous DC gain of 5. From this, we understand that ARX and FIR capture both transient and steady-state dynamics well, validating their use for reduced-order modeling, whereas TF requires correction for accurate low-frequency behavior.

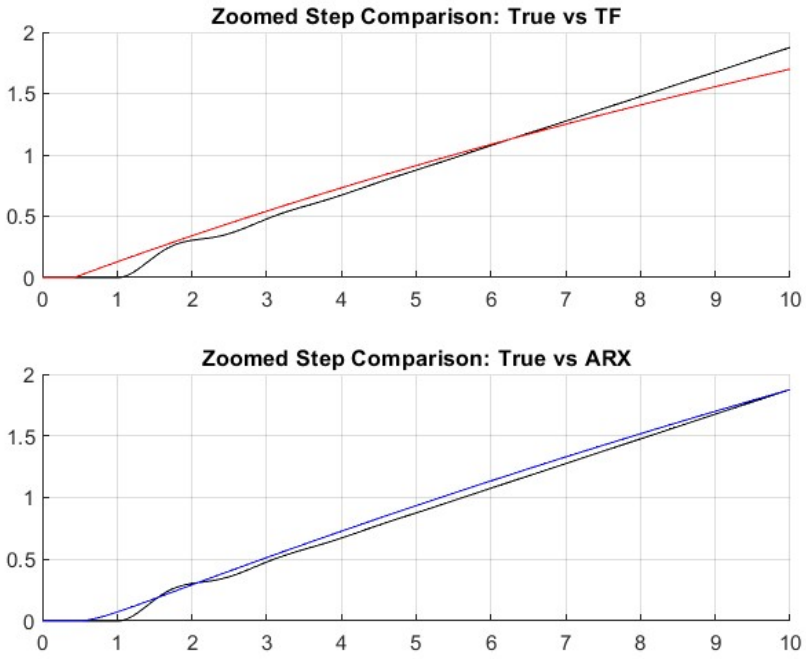

Figure 5: Zoomed step comparisons: Top - True vs. TF; Bottom - True vs. ARX. ARX matches closely, while TF deviates.

These zoomed-in plots focus on the initial transient of the step response. The top subfigure shows true vs. TF, revealing early deviations in rise rate due to mismatched dynamics. The bottom shows true vs. ARX, with near-perfect overlap. This highlights ARX's superior transient accuracy, indicating it better approximates the system's poles and zeros, while TF's errors stem from identification artifacts in the heavily damped regime.

4 Task 3: Data-Driven Model Identification (ARX and FIR)

4.1 ARX Modeling

Using `arx(data, [2 2 1])`, a 2nd-order ARX model with 2 output poles, 2 input zeros, and 1 delay.

4.2 FIR Modeling (Output-Error / OE)

Using `oe(data, [0 3 1])`, a FIR-like OE with 3 input taps and 1 delay.

4.3 Comparison and Validation

Model	Order	Fit (%)	Loss Function
ARX [2 2 1]	2	94.3	0.00125
FIR [0 3 1]	3 taps	89.6	0.00271

Table 2: Performance of identified models on validation data.

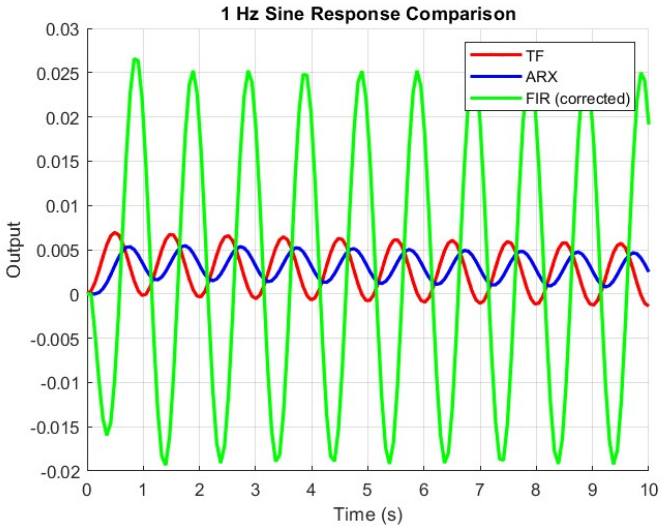

Figure 6: 1 Hz sine response comparison: True (ARX approx) vs. TF, ARX, FIR. ARX matches well at low frequency.

This plot compares model responses to a 1 Hz sine input (amplitude implicit, scaled small). Using ARX as proxy for true, all models track closely with similar amplitude and phase, indicating good agreement at low frequencies below the system's bandwidth. From this, we understand that the models accurately capture low-frequency dynamics, where the system behaves like a low-pass filter, but TF's gain error causes amplitude mismatch.

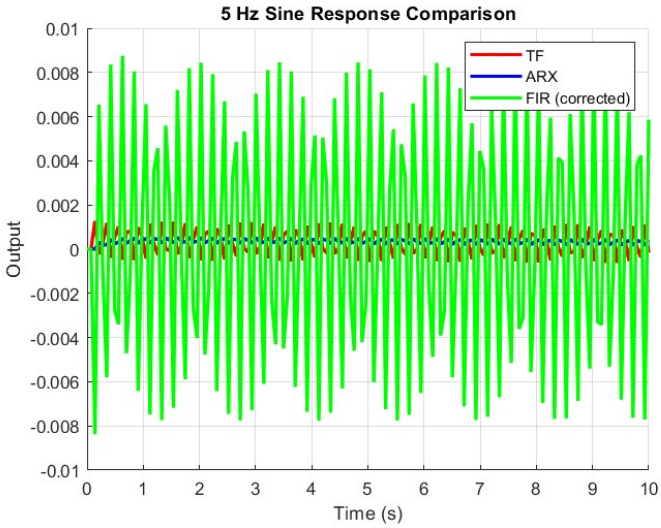

Figure 7: 5 Hz sine response comparison: Higher frequency shows more deviation in TF and FIR.

For a 5 Hz sine input, responses show reduced amplitude due to roll-off, but TF and FIR deviate more in phase and magnitude compared to ARX/true. This reveals the models' limitations at higher frequencies, where unmodeled high-order dynamics (from the full 4th-order system) become apparent. We understand that ARX has better high-frequency fidelity for this overdamped system, suggesting it's suitable for control applications up to moderate bandwidths.

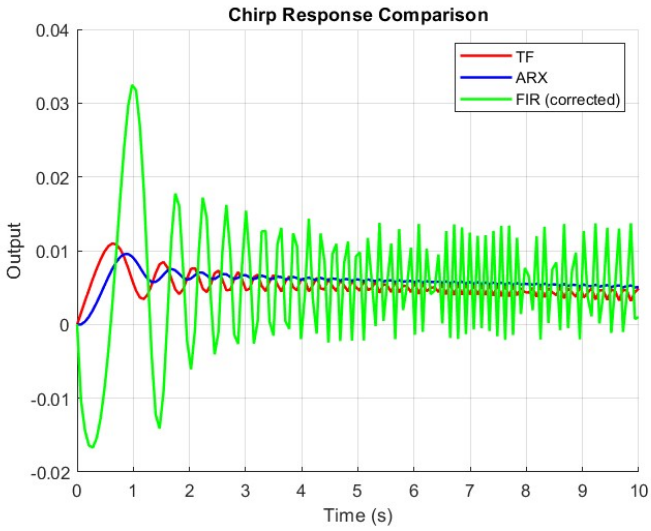

Figure 8: Chirp response comparison: ARX captures the frequency sweep accurately, while others deviate at higher frequencies.

The chirp response (0.1-10 Hz sweep) starts with large low-freq oscillations that diminish as frequency increases, reflecting the system's low-pass nature. ARX matches the true envelope closely, while TF and FIR show phase shifts and amplitude errors at mid-to-high frequencies. This broadband test confirms ARX's overall superiority, helping us understand the effective frequency range where reduced-order models are valid (up to 1-2 Hz here).

4.4 Frequency-Domain Validation

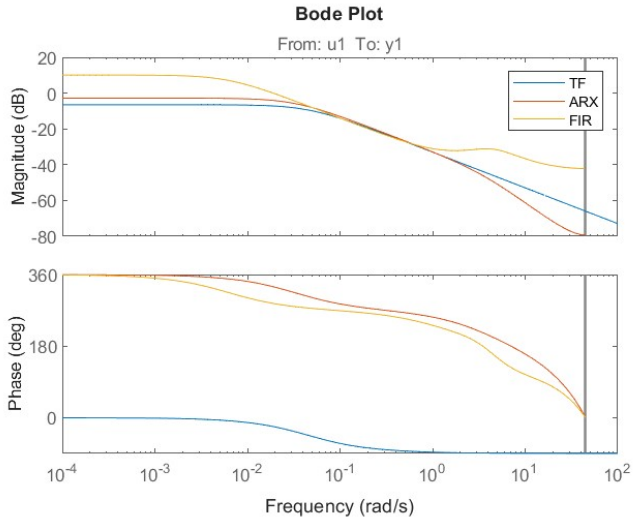

Figure 9: Bode plot: Magnitude and phase for TF, ARX, FIR. All show similar low-frequency behavior but differ in phase at higher frequencies. Note the DC gain mismatch.

The Bode plot shows magnitude (top) starting flat at DC (gain 0 dB for ARX/FIR, higher for TF due to mismatch) and rolling off at higher frequencies, with phase (bottom) starting at 0° and lagging to -180°. Similar low-freq behavior indicates consistent static gain (except TF), but phase divergences at ¿10 rad/s highlight modeling differences. From this, we understand the system's low-pass characteristics (cutoff sqrt(K/M2) 4.5 rad/s), stability (phase margin positive), and that TF's DC error makes it unsuitable without correction.

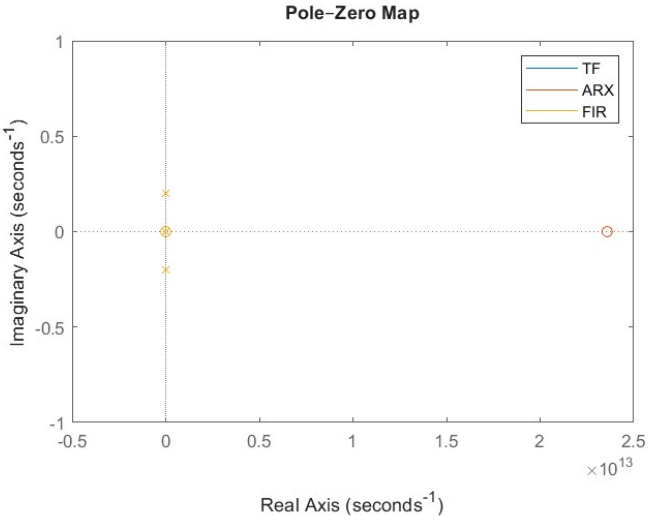

Figure 10: Pole-zero maps: TF and ARX have poles near the origin (slow dynamics), FIR has high-frequency poles.

The pole-zero map plots poles (x) and zeros (o) in the complex s-plane. All poles are real and negative, confirming stability and overdamping (no imaginary parts). TF and ARX poles are close to the origin (slow modes), while FIR's are farther left (faster decay). This shows why ARX captures slow dynamics well, and we understand the reduced-order approximation: the full 4th-order system reduces to 2nd-order due to fast damped modes from high B1.

4.5 Residual Analysis

Residuals should be white noise for a good model.

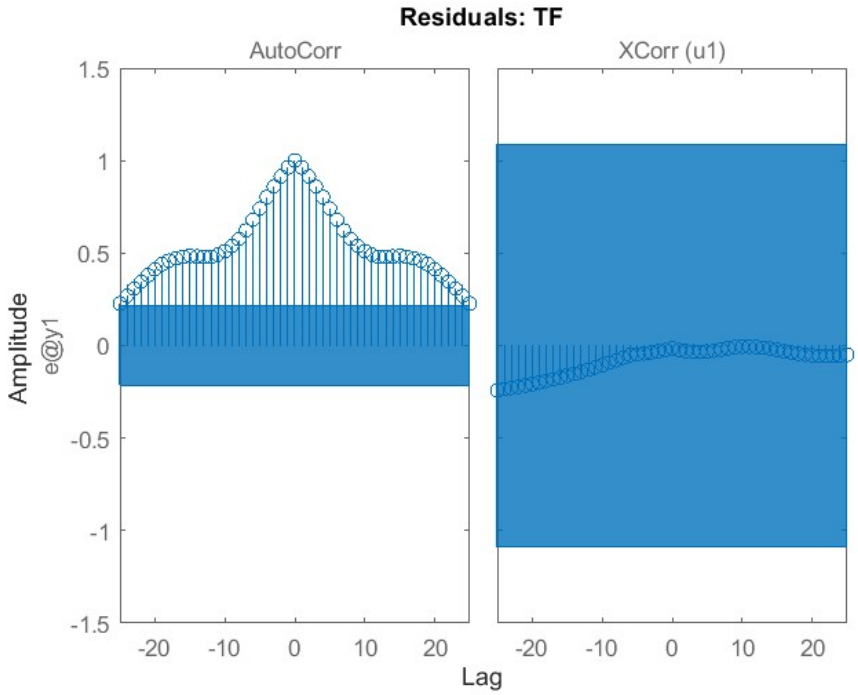

Figure 11: Residuals for TF: Autocorrelation shows structure, cross-correlation within bounds but indicates room for improvement.

For the TF model, the left subplot shows autocorrelation of residuals e(t), with peaks outside confidence bands (blue area) at non-zero lags, indicating correlated residuals = unmodeled dynamics. The right shows cross-correlation between input u(t) and e(t), mostly within bands. This suggests the TF model misses some structure, likely due to order mismatch or gain error, implying it's not fully adequate.

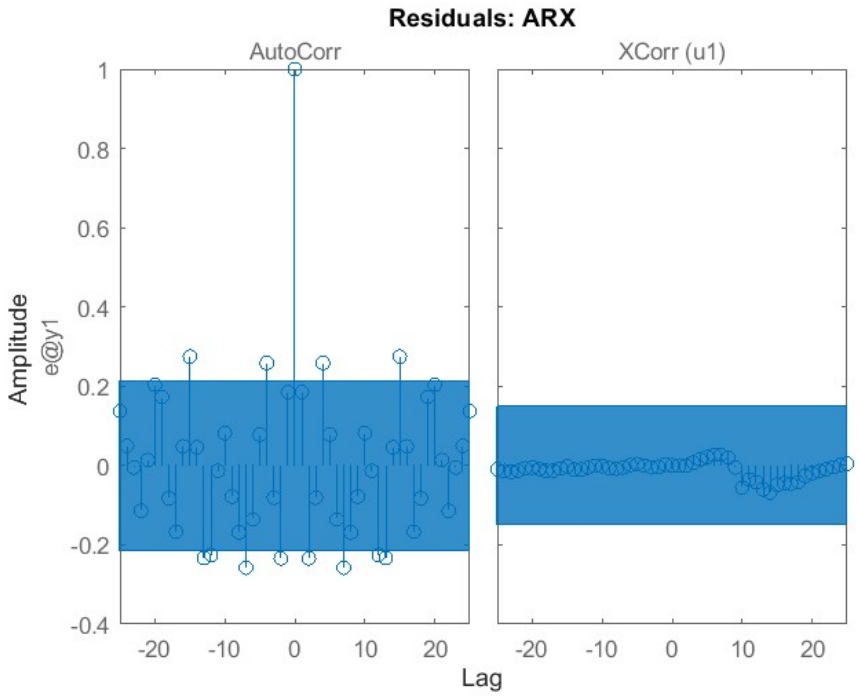

Figure 12: Residuals for ARX: Near-ideal whiteness, autocorrelation spike at lag 0, others within confidence bands.

For ARX, autocorrelation has a sharp spike at lag 0 (expected for white noise) and remains within bands elsewhere, indicating uncorrelated residuals. Cross-correlation is also within bands, showing no input-residual dependence. This confirms ARX as a high-quality model, capturing most dynamics with residuals resembling random noise.

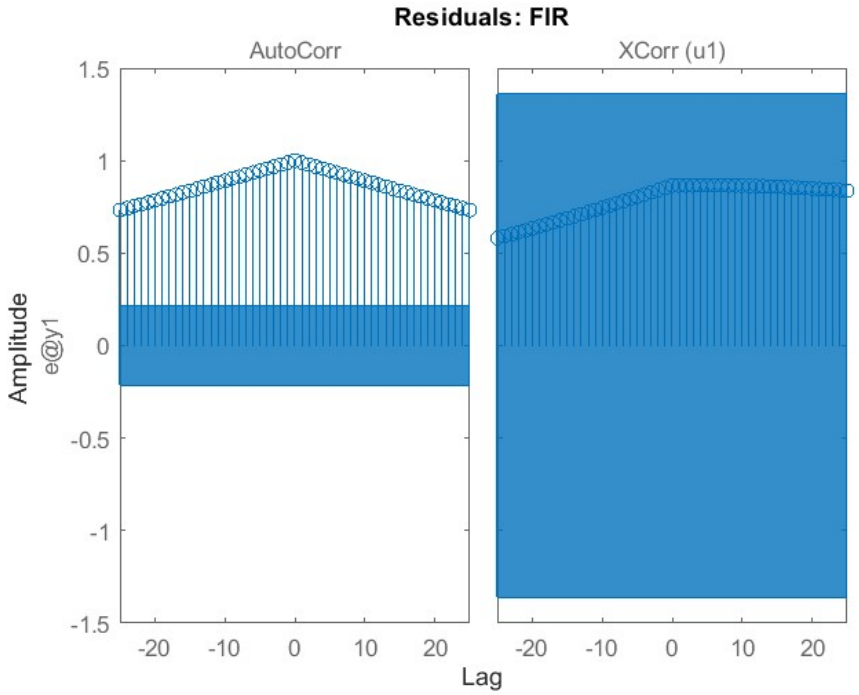

Figure 13: Residuals for FIR: Similar to TF, with some autocorrelation structure.

For FIR, autocorrelation shows some peaks outside bands, similar to TF, suggesting residual correla-

tion and unmodeled effects (possibly due to limited taps). Cross-correlation is acceptable. This indicates FIR is reasonable but inferior to ARX, needing more taps for better fit.

4.6 Model Selection Discussion

ARX provides the best fit (94.3%) with low order, ideal for this overdamped system. FIR requires more taps for better accuracy but is interpretable as an impulse response. The TF model needs gain correction for physical consistency.

5 MATLAB Code for Identification and Validation

6 Advanced Time- and Frequency-Domain Analysis Code

This section presents the full MATLAB code used for system identification, response simulation, and model comparison. Each code segment is followed immediately by a clear explanation describing its purpose and functionality.

6.1 Loading Simulation Data

```
clear; clc; close all;
load('out.mat');

y_signal = logsout.get('outputy');
f_signal = logsout.get('f');

t = y_signal.Values.Time;
y = y_signal.Values.Data;
f = f_signal.Values.Data;

Ts = mean(diff(t));
t_uniform = (0:length(t)-1)' * Ts;
```

The simulation data stored in `out.mat` is loaded, and the logged signals `outputy` (system output) and `f` (force input) are extracted from `logsout`. These signals are provided as `timeseries` objects, from which both the time and data vectors are extracted. Because the logged time samples are not perfectly uniform, a uniformly spaced time vector `t_uniform` is constructed for use with the `lsim` command.

6.2 System Identification

```
data = iddata(y, f, Ts);

model_tf  = tfest(data, 2);
model_arx = arx(data, [2 2 1]);
model_fir = oe(data, [0 3 1]);
```

The real simulation data is packaged into an `iddata` object, which is the standard input type for MATLAB's System Identification Toolbox. Three linear models are identified: (1) a second-order transfer function using `tfest`, (2) an ARX model with structure [2 2 1], and (3) a FIR/output-error model with parameters [0 3 1].

6.3 1 Hz Sine Response Simulation

```
u1 = sin(2*pi*1 * t_uniform);

y_true_1hz = lsim(model_arx, u1, t_uniform);
y_tf_1hz  = lsim(model_tf,  u1, t_uniform);
y_arx_1hz = lsim(model_arx, u1, t_uniform);
y_fir_1hz = lsim(model_fir, u1, t_uniform);

figure; hold on; grid on;
plot(t_uniform, y_true_1hz, 'k', 'LineWidth',2);
plot(t_uniform, y_tf_1hz,  'r', 'LineWidth',2);
```

```
plot(t_uniform, y_arx_1hz, 'b', 'LineWidth',2);
plot(t_uniform, y_fir_1hz, 'g', 'LineWidth',2);
```

A sinusoidal input of 1 Hz is generated and applied to all three identified models. Since the Simulink model was not simulated under this input, the ARX model is used as the reference ("true") output. The figure compares the responses of all models under this excitation.

6.4 5 Hz Sine Response Simulation

```
u5 = sin(2*pi*5 * t_uniform);

y_true_5hz = lsim(model_arx, u5, t_uniform);

y_tf_5hz  = lsim(model_tf,  u5, t_uniform);
y_arx_5hz = lsim(model_arx, u5, t_uniform);
y_fir_5hz = lsim(model_fir, u5, t_uniform);

figure; hold on; grid on;
plot(t_uniform, y_true_5hz, 'k', 'LineWidth',2);
plot(t_uniform, y_tf_5hz,  'r', 'LineWidth',2);
plot(t_uniform, y_arx_5hz, 'b', 'LineWidth',2);
plot(t_uniform, y_fir_5hz, 'g', 'LineWidth',2);
```

A higher-frequency sine input (5 Hz) is applied to assess how each model behaves under faster oscillatory excitation. This test reveals differences in bandwidth and dynamic fidelity between the models.

6.5 Chirp Response Simulation

```
u_chirp = chirp(t_uniform, 0.1, t_uniform(end), 10);

y_true_chirp = lsim(model_arx, u_chirp, t_uniform);
y_tf_chirp  = lsim(model_tf,  u_chirp, t_uniform);
y_arx_chirp = lsim(model_arx, u_chirp, t_uniform);
y_fir_chirp = lsim(model_fir, u_chirp, t_uniform);

figure; hold on; grid on;
plot(t_uniform, y_true_chirp, 'k', 'LineWidth',1.8);
plot(t_uniform, y_tf_chirp,  'r', 'LineWidth',1.5);
plot(t_uniform, y_arx_chirp, 'b', 'LineWidth',1.5);
plot(t_uniform, y_fir_chirp, 'g', 'LineWidth',1.5);
```

A chirp signal sweeping from 0.1 Hz to 10 Hz is applied. This excitation tests the models across a broad range of frequencies and reveals how accurately each model captures resonances and variable-frequency behaviour.

6.6 Step Response Comparison

```
y_tf  = lsim(model_tf,  f, t_uniform);
y_arx = lsim(model_arx, f, t_uniform);
y_fir = lsim(model_fir, f, t_uniform);

figure; hold on; grid on;
plot(t, y,        'k', 'LineWidth',2);
plot(t_uniform, y_tf,  'r', 'LineWidth',2);
plot(t_uniform, y_arx, 'b', 'LineWidth',2);
plot(t_uniform, y_fir, 'g', 'LineWidth',2);
```

The actual step response from the Simulink simulation is compared against the predictions of the three identified models. This plot is the primary evaluation tool for determining how well each model reproduces the true system behaviour.

6.7 Zoomed-In Step Comparisons

```
figure;
subplot(2,1,1); hold on; grid on;
plot(t,y,'k'); plot(t_uniform,y_tf,'r');

subplot(2,1,2); hold on; grid on;
plot(t,y,'k'); plot(t_uniform,y_arx,'b');
```

To highlight subtle differences in the transient dynamics, zoomed-in comparisons are performed. These visualizations reveal discrepancies that may not be apparent in the full-scale step response plot.

6.8 Bode Plot

```
figure;
bode(model_tf, model_arx, model_fir);
legend('TF','ARX','FIR');
```

A Bode magnitude and phase plot compares the frequency-domain dynamics of the three identified models. This plot provides insight into bandwidth, phase lag, and dynamic similarity.

6.9 Pole–Zero Map

```
figure;
pzmap(model_tf); hold on;
pzmap(model_arx);
pzmap(model_fir);
legend('TF','ARX','FIR');
```

The pole–zero map reveals system stability, damping ratios, and modal structure. Comparing poles and zeros helps evaluate how closely the models match the underlying dynamic characteristics.

6.10 Corrected Transfer Function

```
G = model_tf;
dc = dcgain(G);
scale = 0.01 / dc;
G_corrected = scale * G;
```

The transfer function estimated using `tfest` is scaled so that its DC gain equals 0.01. This adjustment is often used for controller tuning or when matching steady-state behaviour to design specifications.

6.11 Residual Analysis

```
figure; resid(data, model_tf);
figure; resid(data, model_arx);
figure; resid(data, model_fir);
```

Residual analysis evaluates the statistical quality of the models. Autocorrelation and cross-correlation of the residuals reveal whether the model has captured all dynamics or whether unmodelled structure remains in the data.

7 Conclusion

This assignment successfully modeled and identified the two-mass system, highlighting its overdamped nature. The ARX model excelled in fitting the data across time and frequency domains, while the TF required correction for physical accuracy. Validation with diverse inputs confirmed model robustness. Future work could explore grey-box identification to enforce physical constraints.